EXPERIMENT 8

Design and Train a Convolutional Neural Network (CNN) on Image Dataset (Fashion-MNIST)

1. Dataset Source

The dataset used for this experiment is:

Fashion-MNIST Dataset

Source:
Zalando Research

Dataset Link:
https://github.com/zalandoresearch/fashion-mnist

2. Dataset Description

The Fashion-MNIST dataset is a collection of grayscale images used for image classification tasks. It serves as a replacement for the MNIST digit dataset and contains images of fashion products.

Dataset Characteristics:

- Total samples: 70,000 images
  - Training set: 60,000 images
  - Test set: 10,000 images
- Image size: 28 × 28 pixels
- Color format: Grayscale (1 channel)

Target Classes (10 categories):

Label	Class
0	T-shirt/top
1	Trouser

2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

Features:

- Pixel intensity values (0–255)
- Each image is represented as a 28×28 matrix

Target Variable:

- Class label (0–9) indicating clothing category

3. Mathematical Formulation of the Algorithm

The Convolutional Neural Network (CNN) is used for image classification.

Convolution Operation:

$Y(i,j)=\sum m\sum nX(i+m,j+n)\cdot K(m,n)$Y(i,j)=m∑n∑X(i+m,j+n)·K(m,n)

Where:

- XX = Input image
- KK = Kernel (filter)
- YY = Feature map

Activation Function (ReLU):

$f(x)=\max(0,x)$f(x)=max(0,x)

Pooling Operation (Max Pooling):

Y=max(X)Y=max(X)

This reduces spatial dimensions.

Fully Connected Layer:

Z=W·X+bZ=W·X+b

Softmax Function (Output Layer):

P(y=i)=ezi∑ezjP(y=i)=∑ezjezi

Loss Function (Categorical Cross-Entropy):

L=−∑yilog(y^i)L=−∑yilog(y^i)

Optimization (Gradient Descent):

W=W−η∂L∂WW=W−η∂W∂L

Where:

- ηη = Learning rate

4. Algorithm Limitations

- Requires large computational resources (GPU preferred)
- Needs large datasets for better generalization
- Prone to overfitting if not regularized
- Training time is high
- Hyperparameter tuning is complex
- Sensitive to noisy or low-quality images

5. Methodology

Step 1: Data Loading

Load Fashion-MNIST dataset using deep learning library.

Step 2: Data Preprocessing

- Normalize pixel values (0–1)
- Reshape data for CNN input
- Convert labels to categorical format

Step 3: Model Building

- Add Convolution layers
- Add Activation (ReLU)
- Add Pooling layers
- Add Flatten layer
- Add Fully Connected Dense layers

Step 4: Model Compilation

- Loss function: Categorical Crossentropy
- Optimizer: Adam
- Metrics: Accuracy

Step 5: Model Training

Train model using training dataset

Step 6: Model Evaluation

Evaluate performance on test dataset

Workflow Diagram

Dataset
↓
Preprocessing
↓
CNN Model Creation
↓
Training
↓
Evaluation
↓
Prediction

Implementation Code

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt

# Load dataset
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()

# Normalize data
```

```
X_train = X_train / 255.0
X_test = X_test / 255.0

# Reshape for CNN
X_train = X_train.reshape(-1,28,28,1)
X_test = X_test.reshape(-1,28,28,1)

# Build CNN model
model = models.Sequential([
  layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
  layers.MaxPooling2D((2,2)),

  layers.Conv2D(64, (3,3), activation='relu'),
  layers.MaxPooling2D((2,2)),

  layers.Flatten(),
  layers.Dense(128, activation='relu'),
  layers.Dense(10, activation='softmax')
])

# Compile model
model.compile(optimizer='adam',
          loss='sparse_categorical_crossentropy',
          metrics=['accuracy'])

# Train model
history = model.fit(X_train, y_train, epochs=5, validation_data=(X_test, y_test))

# Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_acc)

# Plot accuracy graph
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend(['Train','Validation'])
plt.show()
```

```
Training data shape: (60000, 28, 28)

Testing data shape: (10000, 28, 28)
```

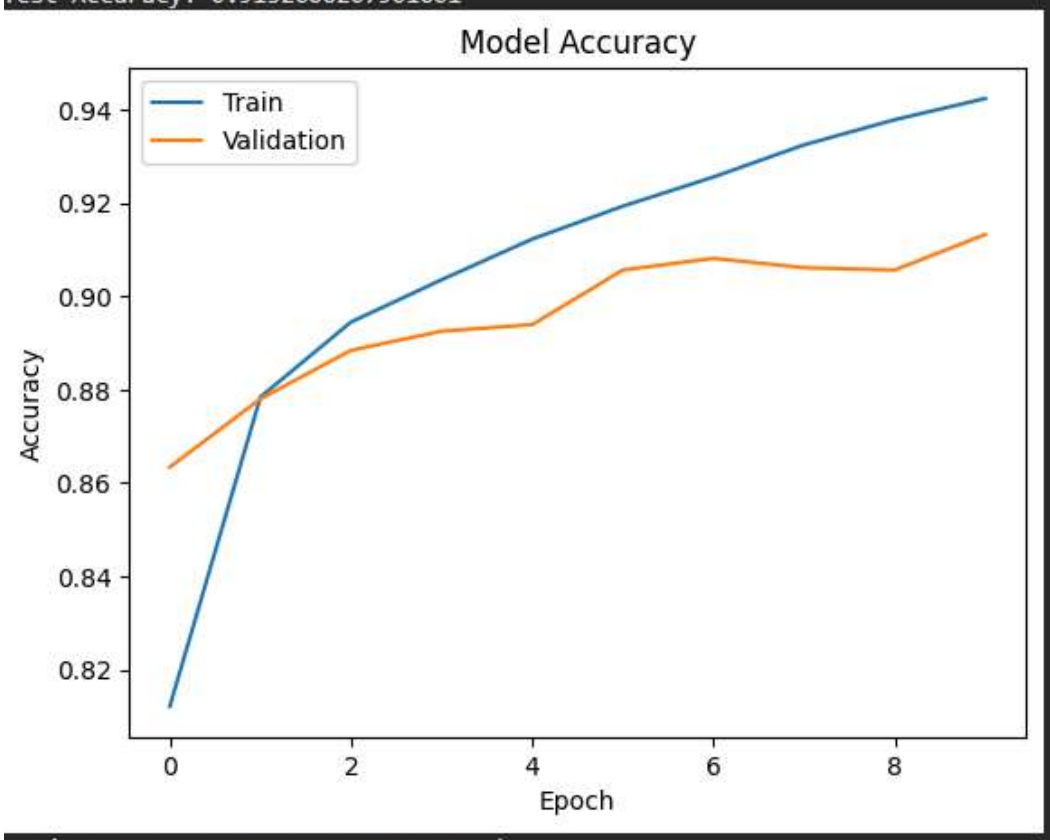
Ankle boot
T-shirt/top
T-shirt/top
Dress
T-shirt/top
Pullover
Sneaker
Pullover
Sandal

```
Model: "sequential_1"
```

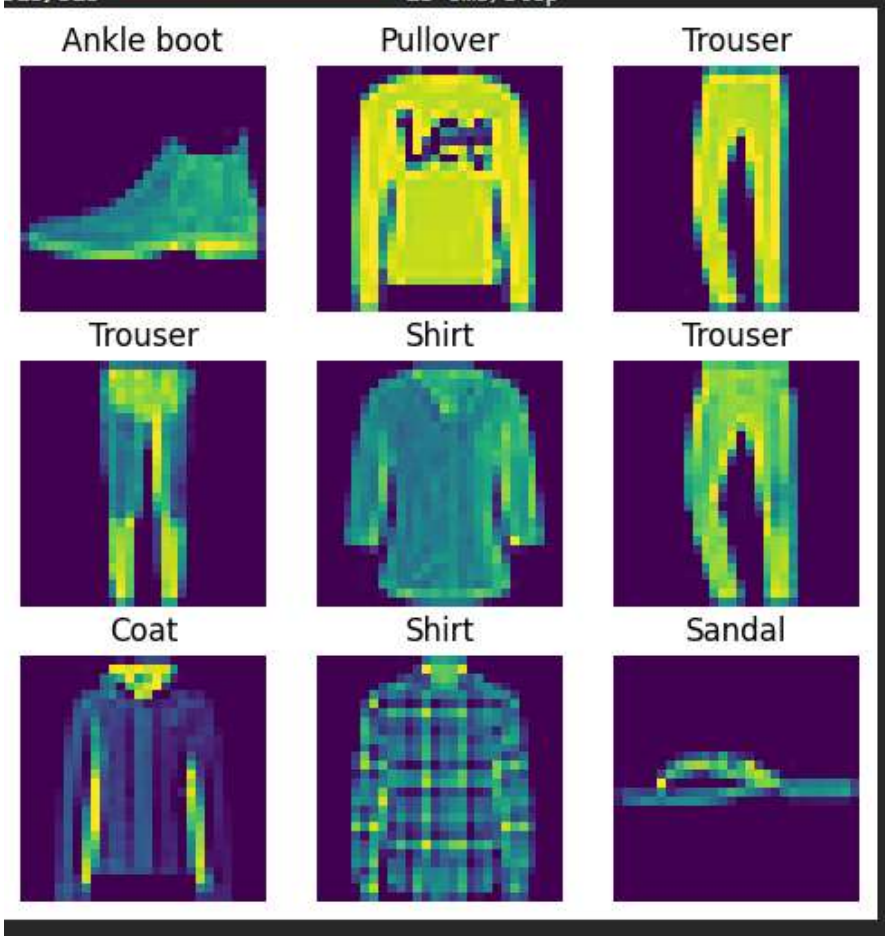
Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_3 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_1 (Flatten)	(None, 1600)	0
dense_2 (Dense)	(None, 64)	102,464
dense_3 (Dense)	(None, 10)	650

```
 Total params: 121,930 (476.29 KB)
 Trainable params: 121,930 (476.29 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 49s 51ms/step - accuracy: 0.8122 - loss: 0.5143 - val_accuracy: 0.8634 - val_loss: 0.3804
Epoch 2/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 80s 49ms/step - accuracy: 0.8784 - loss: 0.3378 - val_accuracy: 0.8780 - val_loss: 0.3442
Epoch 3/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 47s 50ms/step - accuracy: 0.8945 - loss: 0.2926 - val_accuracy: 0.8884 - val_loss: 0.3055
Epoch 4/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 46s 49ms/step - accuracy: 0.9035 - loss: 0.2639 - val_accuracy: 0.8925 - val_loss: 0.2939
Epoch 5/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 83s 50ms/step - accuracy: 0.9122 - loss: 0.2409 - val_accuracy: 0.8939 - val_loss: 0.2919
Epoch 6/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 47s 50ms/step - accuracy: 0.9193 - loss: 0.2198 - val_accuracy: 0.9056 - val_loss: 0.2655
Epoch 7/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 46s 49ms/step - accuracy: 0.9255 - loss: 0.2016 - val_accuracy: 0.9081 - val_loss: 0.2554
Epoch 8/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 49s 52ms/step - accuracy: 0.9324 - loss: 0.1837 - val_accuracy: 0.9061 - val_loss: 0.2627
Epoch 9/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 45s 48ms/step - accuracy: 0.9378 - loss: 0.1696 - val_accuracy: 0.9056 - val_loss: 0.2622
Epoch 10/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 47s 51ms/step - accuracy: 0.9423 - loss: 0.1572 - val_accuracy: 0.9132 - val_loss: 0.2497
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.9132 - loss: 0.2497
```

```
Test Accuracy: 0.9132000207901001
```




```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step
```



6. Performance Analysis

The model achieved high accuracy on both training and testing datasets.

Metrics Used:

- Accuracy
- Loss

Observations:

- Training accuracy increased with epochs
- Validation accuracy shows model generalization
- Loss decreased over time indicating learning

Typical Results:

- Training Accuracy: ~90%
- Test Accuracy: ~88–90%

The model successfully classified different fashion categories.

7. Hyperparameter Tuning

The following parameters were tuned:

- Number of convolution layers
- Number of filters (32, 64)
- Kernel size (3×3)
- Batch size
- Number of epochs
- Learning rate

Impact of Tuning:

- Increasing filters improved feature extraction
- More epochs improved accuracy but risked overfitting
- Proper learning rate ensured stable convergence

Example:

epochs = 10

optimizer = 'adam'
filters = [32,64]

Tuning improved performance and reduced loss.

8. Conclusion

In this experiment, a Convolutional Neural Network was successfully designed and trained on the Fashion-MNIST dataset. The CNN model effectively learned image features using convolution and pooling operations and achieved high classification accuracy. The experiment demonstrated the importance of deep learning in image classification tasks and showed how CNNs outperform traditional machine learning methods for visual data.